

1 Objetivos do trabalho

O presente trabalho visa implementar um programa de edição de imagens em C, no qual são sequencialmente aplicados uma série de filtros e texturas a todas as imagens num dado diretório, de forma a simular um efeito de envelhecimento das mesmas. Para o processamento das imagens, recorreu-se à biblioteca gd. Para a primeira fase, converteu-se um programa originalmente sequencial num programa de execução paralela, com o número de threads e meio de ordenação definidos pelo utilizador. Na segunda fase, fez-se a atribuição das imagens ao programa por meio de um pipe.

2 Funcionalidades implementadas

Table 1: Funcionalidades implementadas

	Não implementada	Com falhas	Totalmente correta
Argumentos da linha de comando			x
Leitura dos nomes das imagens			x
Criação da lista			x
Ordenação da lista			x
Criação pastas resultados			x
old-photo-paralelo-A			
Criação de <i>threads</i>			x
Distribuição trabalho pelas <i>threads</i>			x
Verificação ficheiro existentes			x
Produção imagens transformadas			x
Gestão de memória			x
Produção de ficheiro de estatísticas			x
old-photo-paralelo-B			
Criação de <i>threads</i>			x
Envio das imagens para as <i>threads</i>			x
Verificação ficheiro existentes			x
Produção imagens transformadas			x
Gestão de memória			x
Produção de ficheiro de estatísticas			x
Impressão das estatísticas no ecrã			x

Terminação das <i>threads</i>			x
-------------------------------	--	--	---

2.1 Descrição das funcionalidades com falhas

Não aplicável

3 Organização do código e estrutura de dados

Estruturas de Dados:

struct ImageInfo, presente em old-photo-paralelo-A e old-photo-paralelo-B, armazena informação básica de cada imagem:

filename (nome do ficheiro)
fullPathName (caminho completo do ficheiro)
size (tamanho em bytes)

Medição de tempos:

Utilizam-se variáveis do tipo struct timespec para registar o início e fim das diferentes fases (sequencial, paralela, total). Em old-photo-serial são usadas para medir tempos total e sequencial. Em old-photo-paralelo-A e old-photo-paralelo-B existem ainda arrays de inteiros. para medir o tempo de cada thread.

Distribuição de dados pelas threads (old-photo-paralelo-A):

É usado um array de threads (pthread_t *threads) e uma estrutura ThreadArgs para indicar a cada thread qual o intervalo de imagens (startIdx e endIdx) que deve processar.

Distribuição de dados pelas threads (old-photo-paralelo-B):

Recorrem-se a pipes (pipefd[2]) para distribuir os índices das imagens a processar. Cada thread lê do pipe o índice da imagem e chama a função de processamento correspondente. Existe ainda uma thread adicional que fica atenta à entrada da letra no terminal para imprimir o número de imagens já processadas.

Funções Principais:

process_image, comum às versões paralelas, lê a imagem, aplica transformações (contrast_image, smooth_image, texture_image, sepia_image) e retorna.

Funções de gestão de threads

old-photo-paralelo-A: thread_function recebe um intervalo de índices e processa as imagens desse intervalo, registando tempo de início e fim.

old-photo-paralelo-B: thread_function lê do pipe o índice a processar e executa a função de processamento.

Para ordenação, criaram se funções que utilizam qsort e recebem ponteiros para cmp_name (ordenação por nome) ou cmp_size (ordenação por tamanho). Não foram feitas alterações ao modo como as imagens são processadas. O programa evolui de uma versão sequencial que processa de forma simples as imagens (contrast, smooth, texture, sepia) para versões paralelas.

Em old-photo-paralelo-A, o conjunto de ficheiros é dividido entre as threads.

Em old-photo-paralelo-B, introduzem-se pipes para partilhar dinamicamente os índices das imagens e permitir a monitorização do progresso por meio do teclado.

4 Old-photo-parallel-A

4.1 Algoritmo de partição

O algoritmo começa por aceder a `nnFiles` (número total de imagens) e `threadCount` (número de threads). De seguida, define-se `chunk = nnFiles / threadCount` e `remainder = nnFiles % threadCount`. Depois, para cada thread `i`, calcula-se `size = chunk + 1` se `i < remainder` (caso contrário, `size = chunk`). Atribuem-se os índices `[startIdx, endIdx]` com base em `currentIndex` e `size`. Finalmente, atualiza-se `currentIndex += size` para avançar o intervalo a atribuir à thread seguinte. Pseudocódigo ilustrativo:

```
int chunk = nnFiles / threadCount;
int remainder = nnFiles % threadCount;
int currentIndex = 0;
for (int i = 0; i < threadCount; i++) {
    int size = chunk + (i < remainder ? 1 : 0);
    targs[i].startIdx = currentIndex;
    targs[i].endIdx = currentIndex + size;
    currentIndex += size;
}
```

Table 2: Distribuição de 10 imagens por 3 threads

	Imagens a ser processadas por cada <i>thread</i>			
	1ª imagem da <i>thread</i>	2ª imagem da <i>thread</i>	3ª imagem da <i>thread</i>	4ª imagem da <i>thread</i>
<i>thread_0</i>	IST-1.jpg	IST-4.jpg	IST-7.jpg	IST-10.jpg
<i>thread_1</i>	IST-2.jpg	IST-5.jpg	IST-8.jpg	
<i>thread_2</i>	IST-3.jpg	IST-6.jpg	IST-9.jpg	

4.2 Envio de informação para as *threads*

No `old-photo-parallel-A`, o `main` cria cada thread com `pthread_create`, passando como último argumento um ponteiro para uma estrutura `ThreadArgs`. Essa estrutura contém o `threadId`, que identifica cada thread, e `startIdx` e `endIdx`, que definem o intervalo das imagens que a thread irá processar

As threads acedem às seguintes variáveis globais:

images: array global que contém a informação de cada imagem (filename, fullPathName, size). Cada thread lê apenas as posições correspondentes ao seu intervalo (entre `startIdx` e `endIdx`).

inTextureImage: ponteiro global para a imagem de textura (`paper-texture.png`), utilizada para aplicar o efeito de textura a cada imagem.

outputDir: string global com o caminho do diretório onde as imagens resultantes serão escritas. Cada thread concatena o nome do ficheiro de saída a este caminho.

threadStartTimes e **threadEndTimes**: arrays globais onde cada posição corresponde a uma thread, permitindo que cada registo o próprio tempo de início e fim de execução de cada thread.

5 Old-photo-parallel-B

No old-photo-parallel-B, a distribuição das imagens é feita de forma dinâmica. O main percorre todas as imagens e, para cada índice, escreve-o no pipe. As threads de processamento vão lendo do pipe e processando uma imagem de cada vez, até não haver mais índices disponíveis.

5.1 Descrição das *threads* e *pipes*

- Existem threadCount threads de processamento, que executam a função thread_function, e uma thread adicional para o teclado (keyboardThread).
- Existe apenas um pipe (pipefd[2]). O main escreve no pipefd[1] o índice de cada imagem (inteiro), enquanto as threads de processamento leem esses índices em pipefd[0]. Na thread_function, se a operação de leitura não devolver o número de bytes esperado, significa que já não há mais dados no pipe, e a thread termina.
- A thread do teclado (keyboard_function) está continuamente à escuta de um comando (por exemplo, a tecla S) e imprime o número de imagens já processadas, acedendo a uma variável global que contabiliza as imagens finalizadas.

5.2 Variáveis globais

- images: array global do tipo ImageInfo, onde cada posição descreve uma imagem (nome do ficheiro, caminho completo, tamanho). As threads usam o índice lido do pipe para aceder ao elemento correspondente e processar a imagem.
- inTextureImage: ponteiro global para a imagem de textura (paper-texture.png). As threads usam-na para aplicar o efeito de textura.
- outputDir: string global com o nome do diretório onde se escrevem as imagens finais.
- processedCount: variável global que conta quantas imagens foram já processadas. É protegida por um mutex, para evitar condições de concorrências (processedCountMutex).

5.3 Terminação das threads

O main escreve todos os índices das imagens no pipe. Quando as threads de processamento tentam ler mais dados do pipe e a leitura não retorna o tamanho pretendido, saem do ciclo de processamento e dá-se a pthread_exit (implicitamente, ao retornar da função thread_function).

A thread do teclado termina quando a variável global done é colocada a 1 (no final do programa), permitindo que se faça a pthread_join.

6 Resultados

6.1 Descrição do ambiente de execução

Computador 1 (MacBook Pro M3 Max)

- Processador M3 Max
- Número de Cores: 16 cores (12 de desempenho e 4 de eficiência), 16 threads
- Velocidade do Processador: 4.06 GHz
- Memória: 48 GB de microLPDDR5
- Sistema Operativo: macOS 15 Sequoia

Computador 2 (Máquina Virtual UTM)

- QEMU VM, host arm64 a emular x86-64
- Número de Cores: 4 cores, 4 threads
- Velocidade do Processador: 4.06 GHz
- Memória: 8 GB de RAM
- Sistema Operativo: Ubuntu 24.04 LTS

6.2 Nota: devido a problemas de formatação, os gráficos encontram-se anexados em excel.

7 Discussão

7.1 Efeito do número de Cores

Ao utilizar o Dataset-A para avaliar o desempenho nos dois sistemas de teste, constatámos que o speedup aumenta significativamente à medida que se incrementa o número de threads, até se atingir o limite dos núcleos de alto desempenho. No caso do MacBook Pro com M3 Max (PC1), existem 16 núcleos totais (12 de alto desempenho e 4 de eficiência), correspondendo a 16 threads. Os ganhos de velocidade tendem a ser praticamente lineares até ao uso simultâneo dos 12 núcleos de desempenho; a partir daí, os 4 núcleos de eficiência contribuem para algum ganho adicional, mas não tão acentuado. Quando se ultrapassa o total de núcleos físicos (usando todas as 16 threads), os recursos começam a ser partilhados de forma mais agressiva, o que leva a uma ligeira queda na performance.

Na máquina virtual UTM (PC2), o processador emulado disponibiliza 4 núcleos e 4 threads, operando também a 4.06 GHz. O comportamento segue um padrão semelhante, ainda que à escala de apenas 2 núcleos reais. O acréscimo de threads até 2-3 torna o

processamento mais rápido, mas ao chegar às 4 threads, que já exploram totalmente o hardware emulado, os benefícios adicionais tornam-se menos evidentes.

Assim, a melhor relação entre número de threads e aceleração do processamento surge quando se utilizam tantos threads quanto os disponíveis em HW, pois o uso de mais threads do que disponíveis em hardware começa a gerar partilha de recursos, sobrecarga e menor ganho marginal.

7.2 Efeito da ordem das imagens

No caso do old-photo-parallel-A, o modo de distribuição das imagens baseia-se apenas em atribuir a cada thread um conjunto fixo de ficheiros. Se as imagens tiverem tempos de processamento muito variados e forem mal distribuídas (por exemplo, pela ordenação por nome no Dataset-B), algumas threads podem ficar sobrecarregadas com ficheiros mais demorados, enquanto outras recebem ficheiros mais rápidos, terminando mais cedo. Este desequilíbrio aumenta o tempo total de execução e reduz o speedup.

Já a ordenação por tamanho tende a melhorar essa distribuição ao intercalar imagens de maior e menor duração. No entanto, dependendo da proporção de imagens grandes e pequenas, a estratégia de ordenação pode não ser suficiente para garantir uma distribuição equitativa, mas, em geral, a ordenação por tamanho oferece maior probabilidade de balancear as cargas entre as threads, pois reduz a possibilidade de uma única thread ficar “presa” a um conjunto de imagens muito pesadas.

7.3 Old-photo-parallel-A vs old-photo-parallel-B

O old-photo-paralelo-A pode alcançar elevadas taxas de speedup quando as imagens são bem distribuídas ou quando o conjunto de dados não é muito heterogéneo. No entanto, em cenários com enorme disparidade de tamanhos e tempos de processamento (como no Dataset-B), pode ocorrer forte desbalanceamento entre threads, levando a períodos de inatividade para algumas delas e prolongando o tempo total.

O old-photo-paralelo-B, por sua vez, utiliza um esquema dinâmico, em que cada thread lê os índices das imagens de um pipe e, quando termina, obtém outro índice. Isto diminui a probabilidade de uma única thread ficar sobrecarregada com tarefas demasiado pesadas. Contudo, há uma sobrecarga adicional associada à leitura/escrita no pipe e à sincronização necessária, o que pode reduzir ligeiramente o speedup quando comparado com uma divisão de tarefas perfeita no old-photo-paralelo-A.

Em muitos cenários, a abordagem dinâmica do old-photo-paralelo-B equilibra melhor as tarefas, resultando em tempos de execução mais consistentes, sobretudo quando há muita variabilidade nos ficheiros. Já o old-photo-paralelo-A pode ser ligeiramente mais rápido em casos ideais, desde que as imagens sejam semelhantes ou que a ordenação garanta uma divisão equilibrada.

8 Conclusão

A realização deste projeto demonstrou a importância de uma estratégia adequada de paralelismo, seja por divisão estática ou atribuição dinâmica das tarefas. Observou-se que

o número de núcleos de desempenho disponível nos sistemas tem forte impacto na escalabilidade: a partir do momento em que os núcleos reais se esgotam, o ganho adicional começa a ser limitado. Além disso, a forma como as imagens são ordenadas (por nome ou por tamanho) pode gerar diferenças acentuadas no balanceamento das cargas de trabalho, sobretudo no old-photo-paralelo-A.

Comparando as duas abordagens paralelas, concluímos que o old-photo-paralelo-B tende a distribuir melhor o processamento em cenários de grande diversidade de ficheiros, ao passo que o old-photo-paralelo-A pode ser mais eficiente quando as imagens são mais homogêneas ou a ordem de entrada já garante um bom balanceamento. A experiência adquirida na implementação e avaliação destas soluções mostrou como otimizar a divisão de tarefas, a coordenação entre threads e o uso dos recursos de hardware, reforçando a compreensão sobre paralelismo e desempenho em sistemas multi-core.

9 Anexo

Computador 1:

Hardware Overview:

Model Name:	MacBook Pro
Model Identifier:	Mac15,8
Model Number:	Z1AZ00230LL/A
Chip:	Apple M3 Max
Total Number of Cores:	16 (12 performance and 4 efficiency)
Memory:	48 GB
System Firmware Version:	11881.1.1
OS Loader Version:	11881.1.1
Serial Number (system):	FMXY215W54
Hardware UUID:	4F147428-88D4-50D4-AC19-90D923E6396C
Provisioning UDID:	00006031-001470DE3C78001C
Activation Lock Status:	Disabled

MacBook Pro

14-inch, Nov 2023

Chip	Apple M3 Max
Memory	48 GB
Startup disk	Macintosh HD
Serial number	FMXY215W54
macOS	Sequoia 15.0.1

More Info...

Computador 2:

System Details

Hardware Information	Software Information
Model QEMU QEMU Virtual Machine	OS Name Ubuntu 24.04.1 LTS
Memory 8.0 GiB	OS Type 64-bit
Processor	GNOME Version 46
Graphics Software Rendering	Windowing System Wayland
Disk Capacity 68.7 GB	Virtualization QEMU
	Kernel Version Linux 6.8.0-51-generic

```
CPU op-mode(s): 64-bit
Byte Order: Little Endian
CPU(s): 4
On-line CPU(s) list: 0-3
Vendor ID: Apple
Model name: -
Model: 0
Thread(s) per core: 1
Core(s) per socket: 4
Socket(s): 1
Stepping: 0x0
BogoMIPS: 48.00
```